

R Bootcamp 4 - Intro to Data Analysis

Giovanni Castro Irizarry
UCLA M183/M136C

Spring 2025 Week 5

Today we are working with the a generated dataset with `dplyr`. Below the code that generates the dataset below. If the code doesn't run, make sure to install the `tibble` package with `install.packages("tibble")`

```
# Set seed for replication
set.seed(9001)

# Generate dataset of 100 observations of treatment status and 3 variables
s <- c(1:100) # 100 observations
example_data <- tibble(subject = s) # subjects are our 100 observations
example_data <- example_data %>%
  mutate(treatment = rbinom(100, 1, 0.5)) %>% #binomial distribution
  mutate(var1 = 3*treatment + rnorm(100, 4, 0.5),
         var2 = 2*rnorm(100,6,5),
         var3 = treatment + rnorm(100, 3, 6))
```

OLS in R

Linear regression is the process by which we generate a line of best fit (one that minimizes the sum of squared prediction errors) for a given set of data.

In R, we use the `lm()` function to perform regressions. This function consists of (at minimum) a formula parameter and a data parameter. The formula tells R what particular regression to perform, in the form of “Dependent Variable ~ Independent Variable”. The data parameter tells R the dataset to use. Suppose `var1` is the dependent variable. Then the regression is:

```
# Run a regression of var1 on treatment
example_lm <- lm(var1 ~ treatment,
                 data = example_data)

# Output a summary of the results
summary(example_lm)

##
## Call:
## lm(formula = var1 ~ treatment, data = example_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.96598 -0.36008 -0.05646  0.34946  1.57696
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)  4.04158    0.07163    56.42    <2e-16 ***
## treatment    2.92582    0.09933    29.45    <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4963 on 98 degrees of freedom
## Multiple R-squared:  0.8985, Adjusted R-squared:  0.8975
## F-statistic: 867.6 on 1 and 98 DF,  p-value: < 2.2e-16
```

Coefficients and p-values in regression analysis work together to tell you which relationships in your model are statistically significant and the nature of those relationships. The **coefficients** describe the mathematical relationship between each independent variable and the dependent variable. The **p-values** for the coefficients indicate whether these relationships are statistically significant.

- **Coefficients**

The value of the regression coefficient tells you how much the mean of the dependent variable changes given a one-unit shift in the independent variable while holding all the other variables in the regression model constant.

The sign of the coefficient tells you whether there is a positive or negative correlation between the independent variable and the dependent variable. A positive coefficient suggests that as the value of the independent variable increases, the mean of the dependent variable also tends to increase. A negative coefficient suggests that as the independent variable increases, the mean of the dependent variable tends to decrease.

Notice the “estimate column”. That’s the coefficient we are interested in. The value beside treatment (2.92582) is the estimated treatment effect.

- **P-values**

P-values help determine whether the relationship that you observe in your sample also exists in the larger population. The p-value for each independent variable tests the null hypothesis that the independent variable has no statistically significant correlation with the dependent variable. This means that there is insufficient evidence to conclude that there is an effect at the larger population level.

If the p-value is less than the significance level, your sample provides enough evidence to reject the null hypothesis for the larger population. This statistical significance suggests that the changes in the independent variable are associated with the changes in the dependent variable at the larger population level.

On the other hand, if the p-value is greater than the significance level, your sample does not provide sufficient evidence to conclude that there is a statistically significant effect at the larger population level.

Notice the “Pr(>|t|)” column. That’s the p-value we are interested in. The value beside treatment (<2e-16) is the p-value. Generally, we want for our treatment effects to have p-values below .05. Therefore, this low p-value tells us that it is very unlikely that the observed effect is due solely to chance variation in the data - in other words, that our effect is statistically significant.

The t-test

The intuition behind the t-test is that we want to check if two vectors of data are drawn from the same distribution. For our purposes, we want to know if the treatment group is different than the control group. In an experiment, you want for the treatment group to be different from the control group. However, pre-treatment covariates should NOT be different between groups (otherwise there may be confounding).

Let’s see an example: we will t-test `var1`.

```

# Extract var1 from the treated and control groups
treat_var1 <- (example_data %>% filter(treatment == 1))$var1
control_var1 <- (example_data %>% filter(treatment == 0))$var1

# Apply the t.test function
t.test(treat_var1, control_var1)

##
## Welch Two Sample t-test
##
## data: treat_var1 and control_var1
## t = 29.686, df = 96.755, p-value < 2.2e-16
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  2.730201 3.121442
## sample estimates:
## mean of x mean of y
##  6.967400  4.041578

```

The p-value 2.2e-16 is in scientific notation. We can use the following code to convert 2.2e-16 to standard decimal notation.

```
format(2.2e-16, scientific = FALSE)
```

```
## [1] "0.000000000000000022"
```

0.000000000000000022 < 0.05, so the result from the t-test is statistically significant

Let's take a moment to discuss the output. The important statistic that we're interested in is the p-value. This tells us the probability that any observed difference between groups is due to chance. This p-value tells us that there is very little chance that the groups are different, which (assuming `var1` is your dependent variable) means that the treatment is causing a difference between groups. Notice that the significance result agrees with the regression we did earlier. This should always be the case!

Mechanically, the `t.test()` function should take two vectors of observations, one for each group. Notice how the two parameters in my function call above are named for the treatment and control groups.

Let's try the t-test for `var2`

```

# Extract var2 from the treated and control groups
treat_var2 <- (example_data %>% filter(treatment == 1))$var2
control_var2 <- (example_data %>% filter(treatment == 0))$var2

# Apply the t.test function
t.test(treat_var2, control_var2)

##
## Welch Two Sample t-test
##
## data: treat_var2 and control_var2
## t = -0.044796, df = 97.734, p-value = 0.9644
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -3.785089  3.617982
## sample estimates:
## mean of x mean of y
##  12.24952  12.33308

```

Here, the p-value indicates that there is a .96 chance that the difference between groups is due solely to random noise (i.e. there is a very good chance that both vectors of data were drawn from the same distribution). If `var2` is a pre-treatment covariate, this would be evidence that randomization to treatment assignment has worked.

Let's test `var3`.

```
# Extract var3 from the treated and control groups
treat_var3 <- (example_data %>% filter(treatment == 1))$var3
control_var3 <- (example_data %>% filter(treatment == 0))$var3

# Apply the t.test function
t.test(treat_var3, control_var3)

##
## Welch Two Sample t-test
##
## data: treat_var3 and control_var3
## t = 1.0272, df = 92.693, p-value = 0.307
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -1.162862 3.654613
## sample estimates:
## mean of x mean of y
## 5.012791 3.766915
```

The p-value here shows that there is not a statistically significant difference between groups, but this p-value is much lower than the one in the previous test.

Balance Tables

Balance tables allows us to assess balance on covariate distributions. There are different ways to do this.

We are going to be using the `baltest.collect()` function in the `ebal` package. This requires you to install the `Matching` package as well.

There are three steps:

1. We first need to get our data prepared for the balance table making function. The `MatchBalance` function will help you do that. You only need to input two parameters for it to work properly: a formula regressing the treatment on the covariates that you want to make sure are balanced, and a data parameter containing the dataset you are using. Save this result!
2. The next thing we need is the vector of the names of the covariates that we are balancing. I used the `colnames()` function from a few weeks ago with a few elementwise selection options to get them easily.
3. Last, we put everything in the `baltest.collect()` function. The first parameter is the helper object that we made using the `MatchBalance()` function, the second is the vector of covariate names, and the third indicates that we don't want to compute balance after a unit matching procedure. THE LAST PARAMETER SHOULD ALWAYS BE "after = F" FOR THIS TO WORK.

```
# Create helper object
bal_data <- MatchBalance(treatment ~ var2 + var3,
                        data = example_data,
                        print.level = 0)

# Get covariate names
cov_names <- colnames(example_data)[4:5]
```

```
# Create balance table
baltest.collect(matchbal.out = bal_data,
                var.names = cov_names,
                after = F)
```

```
##      mean.Tr  mean.Co      sdiff sdiff.pooled var.ratio    T pval KS pval
## var2 12.249524 12.333077 -0.8403608  -0.8942647 1.3052145 0.9643611  0.448
## var3  5.012791  3.766915 22.4975932  20.6252358 0.7248469 0.3070160  0.240
##      qqmeandiff qqmeddiff qqmaxdiff
## var2 0.04035256 0.03125000 0.1586538
## var3 0.05780449 0.04807692 0.1923077
```

Notice how the “T pval” column in the balance table output is the same as the individual values for the t-tests we performed above. We also have plenty of other summary statistics that we can use if the situation demands it, but we won’t discuss them at length.

I have created [R Practice 2](#) for more practice with running regressions, t-tests, and balance tables along with data visualizations. You can work through them and let me know if you have any questions.